

Justificaciones de DOM

Representante

Decidimos separarla porque un representante es simplemente un contacto administrativo de una empresa. No tenía sentido obligarlo a registrarse como "Donante", ya que eso mezclaría a los empleados que solo van a firmar un papel con las personas que realmente aportan a la ONG.

Donante (Persona Humana y Juridica)

En lugar de armar un árbol de herencia complejo, hicimos que tanto las personas humanas como jurídicas tengan un perfil de Donante. La razón principal es centralizar todo el historial de donaciones en un solo lugar, evitando tener que repetir el mismo código o listas en ambas clases.

Medio de Contacto y Contacto Predeterminado:

Para saber a dónde enviarle las notificaciones principales al usuario, simplemente agregamos un atributo que apunte de forma directa al contacto elegido dentro de su lista. La razón de esto es mantener el diseño lo más simple y directo posible.

Bienes, Categorías y Subcategorías:

En vez de crear muchas clases distintas (como "Bien Usado" o "Bien Perecedero"), unificamos todo en una sola clase Bien con todos los campos posibles. La razón principal es evitar enredos lógicos: dejamos que la clase Categoría decida qué datos son obligatorios pedir. Si un dato no aplica, simplemente queda vacío.

Modelado de la Segmentación de Donaciones:

Creamos la clase RegistroDonacion para representar el momento exacto en que entran las cajas al depósito. Su función principal es recibir toda la mercadería mezclada y encargarse de dividirla automáticamente en objetos Donacion separados, dejándolos correctamente agrupados por subcategoría para facilitar su futura entrega.

Modelado del Estado de la Donación

Para gestionar el estado logístico de la donación, decidimos utilizar un historial de estados (clase CambioEstado con su fecha respectiva) en lugar de un único atributo que se sobrescriba. La razón principal es garantizar la trazabilidad total: necesitamos poder auditar el ciclo de vida completo de la caja y saber exactamente cuándo ocurrió cada movimiento, sin perder los datos pasados.

Modelado del Componente Notificador

Para resolver el envío de notificaciones, optamos por aplicar el patrón Strategy puro, definiendo el método notificar(mensaje) en la interfaz MedioContacto y dejando que cada clase hija (Email, Telefono, WhatsApp) resuelva su propia implementación.

Justificaciones de CU

Iniciar Sesión:

Decidimos no meter el CU "Iniciar Sesión" en el diagrama general para no hacer bulto y que el gráfico se entienda rápido a simple vista. Igual, como el tema de la seguridad y el login es súper importante, lo dejamos anotado como una precondition obligatoria en la documentación detallada de cada caso de uso.

Registrar Donante y Registrar Entidad Beneficiaria:

Preferimos separar estos dos en casos de uso distintos dentro del diagrama en vez de meter todo en un mismo "Registrar Usuario". Básicamente lo hicimos porque los datos que manejamos no tienen nada que ver: a los donantes les validamos el DNI, género o cosas de su empresa, mientras que las entidades son más del tipo escuela o comedor y nos importan sus datos y quiénes son sus representantes.

Exponer Notificación:

Separamos la lógica de los mensajes en el CU "Exponer Notificación", el cual termina siendo incluido por "Importar Donantes", "Registrar Donante" y "Registrar Entidad Beneficiaria". A su vez, lo conectamos con un actor secundario ("Servicio Externo de Notificación") para representar la integración con plataformas de Email, SMS y WhatsApp.